

STORYVERSE

Master Product Requirements Document

Version: 1.0

Status: Hackathon MVP planning document

Purpose: Single source of truth for the Storyverse team

1. EXECUTIVE SUMMARY

Storyverse is a collaborative visual-storytelling platform that combines Git-style version control with an AI-assisted creative workspace. It allows one creator or a team to build a visual story, create alternate branches, fork another creator's public story, compare changes, and merge selected ideas back into a main canon.

The product is intentionally focused on visual storytelling rather than long-form novel writing. Creators work with scenes, comic panels, character designs, storyboards, visual timelines, story branches, and world relationships. AI is embedded throughout the workflow as a creative collaborator, continuity reviewer, merge assistant, character designer, reader simulator, and canon-memory system.

The simplest description is:

"A collaborative visual-storytelling platform with GitHub-style branching and merging, AI-powered continuity, and multimodal creation tools."

The stronger positioning is not "GitHub for stories" as the full identity. GitHub is the mental model for explaining version control. Storyverse's real identity is a collaborative operating system for visual storytelling.

2. THE CORE PROBLEM

Visual storytellers currently work across disconnected tools. A team may write scenes in one application, design panels in another, exchange images in chat, track feedback in comments, and manually remember which version is considered canon. This creates several recurring problems:

- Story continuity breaks across drafts and contributors.
- Alternate ideas are lost because creators overwrite one another's work.
- Teams cannot easily see why a creative decision was made.
- Character appearance drifts between generated images.
- New collaborators need too much time to understand the project.
- It is difficult to compare two versions of a scene visually.
- Story branches become confusing when multiple people explore different directions.
- Traditional version-control systems are designed for code, not visual narrative assets.

- AI story tools usually generate content but do not support collaborative structure, review, or provenance.
- Small teams lack a unified place to create, organize, review, and evolve multimedia stories.

Storyverse solves these problems by treating a story as a living, visual, versioned system rather than a single document.

3. PRODUCT VISION

Storyverse should enable creators to understand a story at a glance, experiment safely, collaborate without destroying one another's work, and use AI without surrendering creative control.

The long-term vision is a platform where visual stories behave like living projects:

- Every scene has history.
- Every branch has a purpose.
- Every character has a stable visual identity.
- Every major creative decision can be explained.
- Every contributor can experiment without damaging canon.
- Every AI-generated change is reviewable and reversible.
- Every story world can be explored through visual graphs, timelines, panels, and relationships.

4. TARGET USERS

Primary hackathon target:

Small teams creating comics, graphic narratives, animated shorts, visual story pitches, and interactive story concepts.

Secondary users:

- Independent comic creators
- Animation and film students
- Indie game narrative teams
- Storyboard artists
- Webtoon creators
- Creative-writing clubs working visually
- Fan-fiction and alternate-universe communities
- Small media teams developing pitches

For the MVP, the team should choose one primary persona and use that persona throughout the demo. The strongest choice is a small comic or animated-short team because the workflow naturally includes characters, scenes, panels, branching, collaboration, and visual consistency.

5. PRODUCT POSITIONING

Recommended pitch:

“Storyverse is a collaborative visual-storytelling platform where teams branch, fork, review, and merge story ideas while AI protects continuity and visual consistency.”

Supporting analogy:

“It uses GitHub-style version control for visual stories.”

Avoid leading with:

- “An AI story generator”
- “Canva plus GitHub”
- “A chatbot for writing”
- “A tool that automatically makes stories”

Those descriptions make the project sound crowded or unfocused. The central differentiator is structured collaboration around evolving visual narratives.

6. KEY DESIGN DECISIONS AND HOW THE IDEA EVOLVED

6.1 Rename from StoryBind to Storyverse

The product name was changed from StoryBind to Storyverse. Storyverse better communicates branching worlds, alternate timelines, shared universes, and visual narrative exploration.

6.2 Remove the novel-writing focus

The original idea included novel writing. That was removed to keep the project unified and less ambitious. Storyverse is now centered on visual storytelling: comic panels, storyboards, visual scenes, character designs, alternate timelines, and multimedia story experiences.

6.3 Keep GitHub as a mechanism, not the brand identity

GitHub provides useful concepts—branches, forks, commits, pull requests, merges, diffs, and project history. However, Storyverse should not feel like a developer tool with story features attached. The interface and language should remain creative and visual.

6.4 Use two different graph systems

Storyverse should include two graphs with different purposes:

Story Branch Tree:

A tree-like visualization where the main trunk represents canon and alternate timelines branch outward. This is the version-history and collaboration view.

Story World Knowledge Graph:

An Obsidian-inspired network showing relationships among characters, locations, objects, factions, events, and themes. This is the story-world understanding view.

The branch tree explains how the story changed. The knowledge graph explains how the story world is connected.

6.5 Make AI a teammate, not a generate button

AI should participate across the workflow: reviewing, checking, explaining, comparing, suggesting, preserving canon, and helping creators make decisions. Image generation is important, but it should not be the product's only AI feature.

6.6 Build one polished end-to-end workflow

The MVP should demonstrate a believable sequence:

Create a story project → upload or generate a character → lock the character design → create scenes or panels → branch the story → let a collaborator change a branch → run AI review → compare changes → merge selected changes → update the branch graph and canon.

7. CORE PRODUCT WORKFLOW

Step 1: Create a Storyverse project

A creator starts with a title, visual style, short premise, target format, and optional world description.

Step 2: Build or upload a character

The creator uploads a hand-drawn sketch, digital sketch, reference image, or text description.

Step 3: Refine and lock the character

AI helps refine the character through conversational edits. Once approved, the creator saves a locked character version with a reference sheet, colors, clothing, proportions, expressions, and identifiers.

Step 4: Create visual scenes

The creator builds scenes as visual nodes containing a panel image, setting, characters, dialogue, action, emotional goal, and story purpose.

Step 5: Generate panels with canon context

AI uses the locked character and world assets to generate new visual panels while respecting established style and story facts.

Step 6: Create a branch

A creator or collaborator branches from any scene to explore an alternate decision, tone, outcome, or visual direction.

Step 7: Collaborate on branches

Different contributors can work independently on different timelines without overwriting the main story.

Step 8: Compare branches

A visual diff shows changes in scenes, dialogue, characters, assets, pacing, and consequences.

Step 9: Run AI review

The continuity inspector and merge assistant identify contradictions, missing setup, character inconsistencies, visual drift, timeline problems, and merge conflicts.

Step 10: Merge selected changes

The team approves specific scenes or ideas and merges them into canon. The story tree updates to show the new history.

8. CORE FEATURES

8.1 Story Project Workspace

A central dashboard containing:

- Story title and premise
- Main canon branch
- Recent contributor activity
- Characters and world assets
- Scene canvas
- Branch tree
- Knowledge graph
- AI review status
- Story health indicators

8.2 Scene Canvas

Creators should work with scene cards or comic-panel nodes rather than long documents. A scene can contain:

- Generated or uploaded artwork
- Dialogue or captions
- Characters present
- Location
- Time
- Main action
- Emotional beat
- Story purpose
- Dependencies on earlier scenes
- Contributor and revision history

8.3 Branching Timeline Tree

The main branch acts as canon. Alternate branches represent “what if” paths, experiments, rejected versions, or collaborator proposals.

Useful branch states:

- Draft
- Under review
- Approved
- Merged
- Abandoned
- Archived

Possible visual behavior:

- The canon trunk is visually dominant.
- Alternate branches extend outward like a multiverse timeline.
- Users can zoom, pan, collapse, and inspect branches.
- Selecting a node opens the scene and its revision history.
- Three.js can add depth, motion, particle trails, or cinematic transitions without replacing usability.

8.4 Forking

Creators can fork a public story or template and create an independent version. The fork retains attribution to its source and can display how it diverged.

Forking should support:

- Public or private forks
- Source attribution
- Difference summaries
- Optional requests to contribute an idea back
- Licensing and permission indicators

8.5 Pull-Request-Style Story Reviews

A contributor can propose a set of changes to the main story. A review page presents:

- Summary of changes
- Scenes added, changed, or removed
- Character or world changes
- Continuity warnings
- Comments and discussion
- AI review
- Approve, request changes, or reject
- Merge selected scenes rather than requiring an all-or-nothing merge

The interface may use more creative language, such as “Story Proposal” or “Canon Review,” while preserving the pull-request concept.

8.6 Visual Story Diff

A visual diff should compare two branches or versions:

- Side-by-side panel comparison
- Changed dialogue

- Added or removed scenes
- Character-design changes
- Pacing changes
- Different emotional arcs
- Different outcomes
- AI-generated explanation of why the branches diverged

8.7 Character Studio

Character Studio is one of the strongest product features.

Inputs:

- Hand-drawn sketch
- Digital sketch
- Uploaded image
- Text description
- Combination of image and prompt

Workflow:

- Upload or generate a base character.
- Ask AI to modify clothing, age, expression, pose, hair, colors, or style.
- Compare versions.
- Approve and lock a design.
- Save the character as a reusable project asset.
- Use the locked character in future scene generation.

Character profile:

- Name
- Reference images
- Color palette
- Clothing rules
- Physical traits
- Expressions
- Personality
- Goals
- Relationships
- Knowledge state
- Current story status
- Version history

8.8 Visual Consistency Inspector

Before or after generating a scene, AI checks:

- Character appearance against the locked reference
- Clothing and prop consistency
- Environment continuity

- Time-of-day consistency
- Style consistency
- Character count and placement
- Whether a visual contradicts known story facts

It should explain problems and offer fixes rather than silently regenerating everything.

8.9 Story World Knowledge Graph

An interactive network maps:

- Characters
- Locations
- Objects and props
- Factions
- Events
- Themes
- Relationships
- Secrets and knowledge
- Scene appearances

Example:

Clicking a character shows connected characters, locations visited, objects owned, conflicts, and scenes where the character appears.

8.10 Canon Bible

The canon bible stores approved facts:

- Character identities
- World rules
- Timeline facts
- Locations
- Relationships
- Visual style rules
- Important objects
- Events
- Prohibited contradictions

AI generation and review should retrieve from this canon rather than relying on a single prompt.

8.11 Real-Time Collaboration

Potential collaboration features:

- Live cursors
- Presence indicators
- Scene-level comments
- Assigning scenes to teammates
- Notifications

- Review requests
- Role-based permissions
- Activity history
- Concurrent editing protection

For the hackathon, full Google-Docs-level editing is unnecessary. Presence, comments, ownership, and branch-based collaboration are enough to demonstrate the concept.

9. AI FEATURES

9.1 AI Creative Director

Reviews the overall story and suggests:

- Missing emotional beats
- Pacing improvements
- Stronger visual transitions
- Repeated scene patterns
- Opportunities for contrast
- Better branch points
- Scenes that do not advance the story

The user must decide whether to accept any recommendation.

9.2 AI Continuity Inspector

Checks:

- Timeline contradictions
- Character knowledge errors
- Characters appearing in impossible locations
- Objects disappearing or changing without explanation
- Relationship inconsistencies
- Violations of world rules
- Missing setup or payoff
- Character-design drift

9.3 AI Merge Assistant

When two branches conflict, the assistant:

- Summarizes each branch
- Identifies compatible changes
- Identifies true conflicts
- Suggests multiple merge strategies
- Explains trade-offs
- Creates a preview without modifying canon
- Lets creators merge selected scenes or details

9.4 AI Branch Suggestion

Detects natural decision points and proposes alternate paths:

- A character makes a different choice.
- A hidden truth is revealed earlier.
- A conflict ends differently.
- The visual tone changes.
- Another character becomes the point-of-view character.

9.5 AI Canon Memory

Retrieves approved facts during generation and review. It should distinguish:

- Canon facts
- Branch-specific facts
- Draft ideas
- Rejected ideas
- Unknown information

9.6 AI Reader Simulator

Simulates first-time readers with different profiles and reports:

- Where the story becomes confusing
- Which character motivations are unclear
- Which scenes feel slow
- Which reveals lack setup
- Which branches are more emotionally effective
- What information the reader believes at each point

9.7 AI Dialogue Coach

Suggests improvements while preserving character voice:

- Tone consistency
- Subtext
- Redundancy reduction
- Speech-pattern consistency
- Age or context appropriateness
- Stronger visual storytelling instead of exposition

9.8 AI Style Harmonizer

Helps maintain a consistent style across contributors and generated assets. For the MVP, support a small set of original style presets rather than imitating named living artists or copyrighted studio styles.

9.9 AI Story Summarizer and Onboarding Assistant

Creates:

- Project overview
- Branch summaries
- “What changed since your last visit”
- Character summaries

- Current unresolved decisions
- Contributor change logs

9.10 AI Project Historian

Explains why creative decisions were made by using comments, approved proposals, branch history, and commit messages.

9.11 AI Asset Reuse

Detects existing characters, props, or backgrounds that can be reused or adapted instead of regenerating duplicate assets.

9.12 AI Story Health Dashboard

Possible indicators:

- Continuity status
- Visual consistency
- Unresolved branches
- Character coverage
- Pacing balance
- Reader-confusion risk
- Canon conflicts
- Review readiness

Scores should always include explanations. Avoid presenting subjective creative quality as an absolute fact.

10. MULTIMODAL EXPERIENCE

Storyverse is naturally multimodal because the modalities support one another within one creative workflow.

Core modalities:

- Text: premise, captions, dialogue, notes, summaries
- Images: sketches, character designs, panels, locations, props
- Graphs: story branches and world relationships
- Interaction: editing, branching, reviewing, merging
- Optional voice: spoken scene ideas, notes, narration
- Optional audio: sound cues, mood references, narration
- Optional animation: panel transitions or lightweight animatics

The strongest MVP should prioritize text, images, graphs, and interaction. Voice and audio can be added only if the core flow is already polished.

11. THREE.JS VISUAL DIRECTION

Three.js should add a memorable visual layer without turning the application into a difficult 3D experiment.

Recommended uses:

- A cinematic multiverse branch tree
- Depth and parallax during branch navigation
- Animated paths showing how a branch evolved
- A 3D story-world relationship view
- Smooth camera transitions from the story tree into a scene
- Particles or pulses representing collaboration and updates
- Timeline playback showing branches growing over time

Usability requirements:

- Labels must remain readable.
- Important controls must remain standard 2D interface elements.
- Users need a reduced-motion option.
- The graph must work even if 3D effects are disabled.
- Do not use Three.js for every screen.

12. MVP SCOPE

The MVP should prove one core idea: visual storytelling becomes easier to evolve collaboratively when branches, AI review, and consistent multimedia assets are integrated.

Must-have features:

- Authentication
- Create a story project
- Create or upload one character
- AI-assisted character refinement
- Lock character reference
- Create a small sequence of visual scenes
- Generate at least one panel using project context
- Create a branch from a scene
- Show branch tree
- Let a collaborator or simulated collaborator modify the branch
- Compare branch changes
- Run AI continuity review
- Merge selected changes into canon
- Update the graph and activity history

Strong optional feature:

- A small knowledge graph connecting character, location, prop, and scene

Do not prioritize for the MVP:

- Full novel editor
- Complete video editor
- Full 3D game engine
- Marketplace
- Complex licensing system
- Dozens of AI agents
- Advanced real-time collaborative text editing
- Mobile application
- Full animation production
- Every possible story format

13. RECOMMENDED DEMO STORY

Use a visually clear story with one character, one companion, one important object, and one major decision.

Example:

A young explorer enters a flooded city to recover a glowing compass. At a key scene, the explorer must choose between saving a trapped stranger and pursuing the compass.

Demo branch A:

The explorer pursues the compass.

Demo branch B:

A collaborator branches the story and saves the stranger.

This creates easy-to-understand differences in panels, character relationships, later scenes, and emotional outcome.

14. THREE-MINUTE DEMO FLOW

0:00–0:20 — State the problem

“Visual-story teams lose alternate ideas, overwrite one another’s work, and struggle to keep characters and canon consistent.”

0:20–0:40 — Show the project

Open the Storyverse workspace and display the main story branch, character, and visual scenes.

0:40–1:05 — Character Studio

Upload a rough sketch, refine it with AI, approve it, and lock the character design.

1:05–1:30 — Generate a scene

Create a comic panel using the locked character and canon context.

1:30–1:55 — Branch and collaborate

Create an alternate timeline from a major choice. Show a collaborator changing the branch.

1:55–2:20 — AI review

Run the continuity inspector. It catches a concrete problem, such as the character using an object that was lost in the previous scene.

2:20–2:40 — Compare and merge

Open the visual diff. Use the merge assistant to accept selected changes.

2:40–2:55 — Wow moment

Zoom out to the Three.js branch tree as the approved branch merges into canon and the graph updates.

2:55–3:00 — Closing line

“Storyverse gives visual storytellers the freedom to explore every possible world without losing the story they are building together.”

15. WHY STORYVERSE COULD WIN

15.1 Clear problem

The project solves recognizable collaboration, continuity, and visual-consistency problems.

15.2 Strong differentiation

Most AI storytelling tools generate stories. Storyverse helps teams organize, review, branch, merge, and preserve complex visual stories.

15.3 Natural multimodality

Text, images, graphs, collaboration, and optional audio are connected through one workflow rather than presented as unrelated features.

15.4 Memorable visual demonstration

A multiverse-like story tree, a sketch becoming a consistent character, and a branch merging into canon create multiple strong demo moments.

15.5 Human control

AI proposes, checks, explains, and previews. Creators approve decisions and retain authorship.

15.6 Technical depth

The product combines graph systems, semantic retrieval, image workflows, versioning, collaboration, AI reasoning, and a polished visual interface.

15.7 Practical feasibility

A focused MVP can demonstrate the concept without building a complete production suite.

15.8 Strong IBM-hackathon framing

Storyverse should be presented as an AI-powered system that enhances human creativity and collaboration. IBM Bob's role in planning, coding, testing, debugging, and documenting the project should be visible and traceable.

16. IBM BOB DEVELOPMENT PLAN

Use IBM Bob as the primary development assistant and preserve evidence of its contribution.

Document:

- Architecture planning sessions
- Component generation
- API design
- Database-schema work
- Graph implementation
- Tests generated or improved with Bob
- Debugging sessions
- Accessibility review
- README and documentation support
- Refactoring suggestions
- Exported Bob sessions or development reports when supported

A "Built with IBM Bob" section in the repository should explain:

- What Bob helped create
- What decisions remained human-led
- How Bob accelerated development
- How the team validated Bob-generated work

17. RECOMMENDED TECH STACK

Frontend:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Three.js
- React Three Fiber
- Framer Motion

Graph and canvas:

- React Flow for story branches and node-based editing

- D3.js for custom layouts and transitions
- Cytoscape.js or Sigma.js for the knowledge graph
- Zustand for local application state

Backend and database:

- Next.js server actions or API routes
- Node.js
- Supabase
- PostgreSQL
- Supabase Auth
- Supabase Storage
- Supabase Realtime
- pgvector for semantic retrieval

Collaboration:

- Liveblocks or Supabase Realtime
- Yjs only if true collaborative editing becomes necessary

AI:

- IBM watsonx or Granite models where appropriate for hackathon alignment
- IBM Bob as the primary development tool
- OpenAI-compatible reasoning and image services only where permitted by the competition
- Vision model for sketch and consistency analysis
- Speech-to-text only if voice notes are included
- Embeddings for canon retrieval and semantic search

Media and processing:

- Cloudinary or Supabase Storage for image assets
- Sharp for image processing
- FFmpeg only if lightweight animatics or video export are included

Testing:

- Vitest
- React Testing Library
- Playwright
- Storybook for UI components

Deployment and observability:

- Vercel
- Supabase
- Sentry
- PostHog or simple event analytics

18. SUGGESTED DATA MODEL

Core entities:

- User
- Team
- Project
- ProjectMember
- Branch
- Scene
- SceneVersion
- StoryProposal
- Review
- Merge
- Character
- CharacterVersion
- Location
- Prop
- WorldFact
- Relationship
- Asset
- Comment
- ActivityEvent
- AIReview
- GenerationJob

Important relationships:

- A project has one main canon branch and many alternate branches.
- A branch contains ordered scene references.
- A scene can have many versions.
- Characters, locations, props, and world facts belong to a project.
- AI reviews attach to branches, scenes, proposals, or assets.
- Merges record source branch, destination branch, selected changes, reviewers, and explanation.

19. PRODUCT PRINCIPLES

- Visual first
- Collaboration before automation
- AI suggestions remain reviewable
- Canon is explicit
- Every change has history
- Alternate ideas are preserved
- Generation should use project context
- Creators control style and authorship
- The graph should clarify, not decorate

- The MVP should tell one complete story

20. RISKS AND MITIGATIONS

Risk: Scope becomes too large.

Mitigation: Build only the create → branch → review → merge workflow.

Risk: Generated characters look inconsistent.

Mitigation: Use locked character references, limited poses, repeated seeds or reference workflows, and a consistency checker.

Risk: The interface feels like a GitHub clone.

Mitigation: Use creative terminology, scene canvases, images, and narrative-focused review screens.

Risk: The 3D graph becomes difficult to use.

Mitigation: Keep a functional 2D fallback and use 3D primarily for overview and demo impact.

Risk: AI review produces vague feedback.

Mitigation: Test with specific continuity rules and require evidence from scene data or canon facts.

Risk: The project looks like another AI story generator.

Mitigation: Lead with collaboration, branching, visual diffs, continuity, and merging.

Risk: Too many target users weaken the pitch.

Mitigation: Demo for one clear persona, preferably a small comic or animation team.

Risk: Copyright or imitation concerns.

Mitigation: Use original project assets and original style presets. Avoid marketing the product around copying named artists or studios.

Risk: Public forking creates ownership concerns.

Mitigation: Include attribution, permissions, visibility controls, and licensing metadata in the product concept.

21. TEAM PRIORITIES

Priority 1:

A polished branch tree and scene workflow.

Priority 2:

Character Studio with a locked visual reference.

Priority 3:

A concrete AI continuity check that catches a real contradiction.

Priority 4:

A visual diff and merge interaction.

Priority 5:

A clean three-minute story-driven demo.

Everything else is secondary.

22. SUCCESS METRICS FOR THE HACKATHON MVP

The MVP succeeds if:

- A judge understands the product within twenty seconds.
- A user can create and branch a story without explanation.
- The same character remains recognizable across generated panels.
- AI catches at least one meaningful story inconsistency.
- A branch can be reviewed and merged visibly.
- The final graph makes the project history understandable.
- The full core demo runs reliably in under three minutes.

23. ONE-SENTENCE PITCH OPTIONS

Primary:

“Storyverse is a collaborative visual-storytelling platform where teams branch, review, and merge story ideas while AI protects continuity and visual consistency.”

More emotional:

“Storyverse lets creative teams explore every possible version of a story without losing the world they are building together.”

More technical:

“Storyverse brings Git-style version control, multimodal AI, and visual collaboration to comics, storyboards, and interactive narratives.”

24. FINAL MVP DEFINITION

For the hackathon, Storyverse is not a complete comic-production platform, novel editor, or animation studio.

It is a focused demonstration of one powerful idea:

A visual story can be treated as a collaborative, versioned universe. Creators can upload and lock a character, build visual scenes, explore alternate timelines, collaborate safely on branches, use AI to detect continuity and visual conflicts, and merge the best ideas back into canon.

That is the product the team should build, explain, and demonstrate.